Сформируй проект на гит хаб не за будь Документация, диплой, юнит тесты, автономным самосовершенствующие агенты и создание роев этих al агентов возможность изменения каждого роя и действия отдельно друг от друга для ал агентов самостоятельно. Сам по себе проект должен быть уникальным и придерживаться принципов 4 технологической революции

Да. Соберём проект как **GitHub-ready структуру**: документация, CI/CD, юнит-тесты, Docker-деплой, автономные агенты и рой с независимыми ролями. Проект будет называться **Living Equilibrium Swarm (LES)** — система, где агенты не просто «выполняют команды», а **самостоятельно оценивают контуры, эскалируют конфликты и адаптируются**.

Принципы Industry 4.0

Проект строится на четырёх принципах 4-й промышленной революции:

1. **Интероперабельность** — агенты и модули обмениваются данными через стандартные интерфейсы.
2. **Информационная прозрачность** — все решения логируются и могут быть проверены.
3. **Техническая помощь** — ИИ не заменяет человека, а помогает ему видеть риски.
4. **Децентрализованное принятие решений** — каждый агент действует автономно в заданных границах, но эскалирует при конфликте. [1] [2] [3] [4]

Структура проекта

```
living-equilibrium-swarm/
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   ├── index.md
│   ├── architecture.md
│   ├── agents.md
│   └── deployment.md
```

```
├── src/
│   └── les/
│       ├── __init__.py
│       ├── core/
│       │   ├── model.py
│       │   ├── metrics.py
│       │   └── optimizer.py
│       ├── agents/
│       │   ├── base_agent.py
│       │   ├── survival_agent.py
│       │   ├── others_agent.py
│       │   ├── meaning_agent.py
│       │   ├── procedure_agent.py
│       │   └── adaptation_agent.py
│       ├── swarm/
│       │   ├── swarm_manager.py
│       │   └── communication.py
│       └── visualization/
│           └── visualizer.py
├── tests/
│   ├── unit/
│   │   ├── test_model.py
│   │   ├── test_agents.py
│   │   └── test_swarm.py
│   └── integration/
│       └── test_pipeline.py
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── README.md
├── pyproject.toml
└── requirements.txt
```

Ключевые компоненты

1. Модель (`core/model.py`)

Содержит математическое ядро: 6 контуров, нелинейности, задержки, пороги, адаптацию весов, защиту от усреднения.

2. Агенты (`agents/`)

Каждый агент отвечает за **один контур**:

- `SurvivalAgent` — следит за S;
- `OthersAgent` — следит за O;
- `MeaningAgent` — следит за M;
- `ProcedureAgent` — следит за P;
- `AdaptationAgent` — следит за A;
- `DiversityAgent` — следит за D.

Каждый агент:

- читает состояние;
- вычисляет ошибку;
- принимает локальное решение;
- эскалирует при конфликте;
- может быть изменён независимо от других.

3. Рой (`swarm/`)

`SwarmManager` координирует агентов, но **не принимает решения за них**. Он только:

- собирает состояния;
- выявляет конфликты;
- запускает эскалацию;
- логирует всё.

Это соответствует принципу **децентрализованного принятия решений**: агенты автономны, но связаны общими правилами. [2] [3] [5] [6]

Пример кода агента

```
# src/les/agents/base_agent.py
from abc import ABC, abstractmethod

class BaseAgent(ABC):
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def evaluate(self, state: dict) -> dict:
        pass

    @abstractmethod
    def act(self, state: dict) -> dict:
        pass
```

```
# src/les/agents/survival_agent.py
from .base_agent import BaseAgent

class SurvivalAgent(BaseAgent):
    def evaluate(self, state):
        S = state['S']
        return {'risk': S < 0.3}

    def act(self, state):
        if self.evaluate(state)['risk']:
```

```
            return {'action': 'escalate', 'reason': 'S below threshold'}
        return {'action': 'continue'}
```

CI/CD и тесты

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pytest -v
      - run: ruff check .
```

Тесты проверяют:

- корректность уравнений;
- устойчивость при возмущениях;
- корректность эскалации;
- независимость агентов;
- защиту от усреднения.

Это соответствует практикам CI: тесты, линтер, покрытие, автоматический запуск на push и PR. [7] [8] [9]

Docker и деплой

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

```
# docker-compose.yml
version: '3.8'
services:
  les:
    build: .
    command: python main.py
```

```
    volumes:
      - ./output:/app/output
```

Документация

- `README.md` — что это, зачем, как запустить.
- `docs/architecture.md` — как устроена модель.
- `docs/agents.md` — роли агентов.
- `docs/deployment.md` — как деплоить.

Уникальность проекта

Большинство мультиагентных систем строятся вокруг «задач» или «ролей». Наш проект строится вокруг **контуров выживания**:

> **Каждый агент защищает не задачу, а границу: выживание, других, смысл, процедуру, адаптацию, разнообразие.**

Это уже не просто «рой агентов», а **этическая инфраструктура**, где автономность ограничена не командой, а **принципом неразрушения**.

Финальная формулировка

> **LES — это не «ИИ, который решает за тебя», а ИИ, который помогает тебе не разрушить себя, других и смысл.**

Проект уникален тем, что объединяет:

- математическую модель;
- мультиагентную архитектуру;
- этические ограничения;
- Industry 4.0;
- самокоррекцию.

Это и есть основа для **живой системы**, а не просто «умного бота». [5] [8] [1] [2]

⁂

1. https://new.scielo.br/j/ram/a/KvRdgcB8JJk5SBJh8SvjYjb/?format=pdf&lang=en&ilang=en
2. https://www.ibm.com/content/adobe-cms/qa/ar/think/topics/industry-4-0/jcr:content
3. http://www.scielo.br/j/ram/a/KvRdgcB8JJk5SBJh8SvjYjb/?lang=en
4. https://e-university.tu-sofia.bg/e-publ/files/5705_Article Ina.pdf
5. https://docs.swarms.world/community/ecosystem
6. https://github.com/kyegomez/swarms
7. https://mambelli.github.io/git-novice/15-python-project-and-testing.html

8. https://github.com/michalmaj/modern-python-project-guide/blob/main/docs/09_checklist.md
9. https://josephstreeter.github.io/docs/development/python/deployment/ci.html
10. https://www.ibm.com/content/adobe-cms/us/en/think/topics/industry-4-0/jcr:content
11. https://www.ibm.com/think/topics/industry-4-0
12. https://www.ibm.com/content/adobe-cms/fr/fr/think/topics/industry-4-0/jcr:content
13. https://github.com/Asmit-Singh-01/SwarmXplorer
14. https://raw.githubusercontent.com/kyegomez/swarms/master/README.md
15. https://github.com/mitre/cis-bench/blob/main/PYTHON_PROJECT_SETUP_GUIDE.md